Bases de Datos

Clase 15: Data Science, Pandas y Data Engineering

Hasta ahora

Ya sabemos como hacer un modelo, transformarlo a tablas en ciertas formas, como guardar los datos en una base, cómo hacer consultas, como calcular la complejidad de las consultas, como hacerlas privadas e incluso como usar herramientas externas para ayudarnos…, ¿y ahora qué?

Ahora veremos… **Data Science**

Data Science

Data Science

“La ciencia de datos es un enfoque multidisciplinario para extraer conocimientos prácticos de los grandes y cada vez mayores volúmenes de datos recopilados y creados por las organizaciones actuales. La ciencia de datos abarca la preparación de datos para su análisis y procesamiento, la realización de análisis de datos avanzados y la presentación de los resultados para revelar patrones y permitir a las partes interesadas sacar conclusiones informadas.”

Según IBM

Data Science

Engloba varios conceptos:

- Data Analysis
- Machine Learning
- Deep Learning
- AI
- Computer Vision
- Natural Language Processing

y varios más …

Data Science

Engloba varios conceptos:

- **Data Analysis**

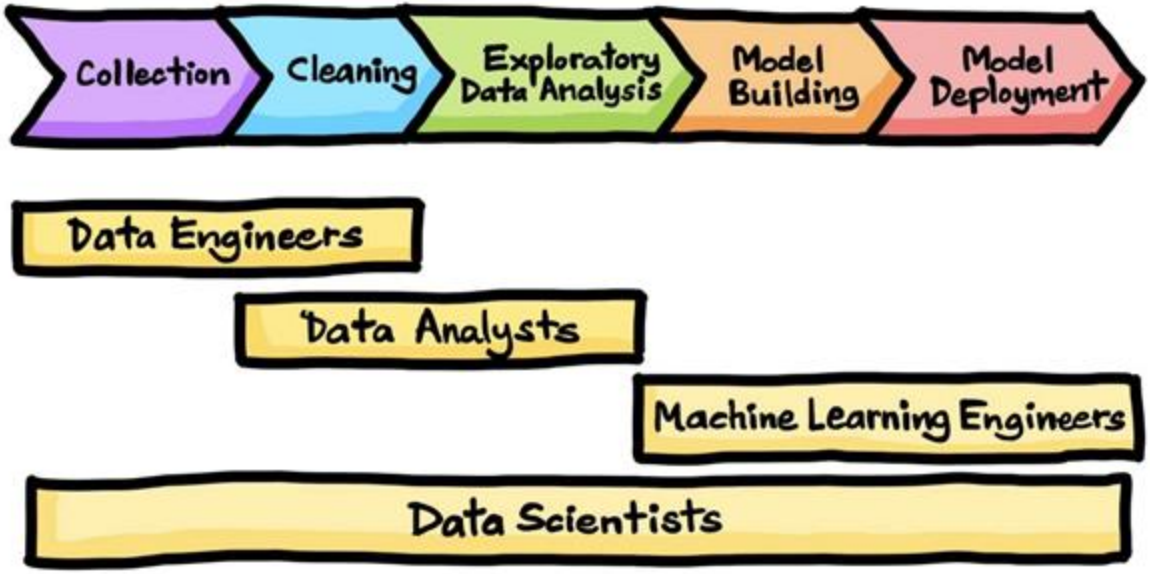
En esta clase veremos la librería más usada en Python para hacer Data Analysis: Pandas 🐼

En general, hay otros cursos que tocan los demás tópicos…

Data Analysis

Es el proceso de inspección, limpieza, transformación y modelamiento de datos que tiene como objetivo extraer información relevante para algún fin en particular.

El proceso de Data Science



En general, se ve así…

Tenemos los datos en una BD

Los cargamos en un entorno para trabajarlos (jupyter notebook, R, Tableau, etc)

Extraemos información importante con la cual se arman reportes, modelos, etc

La librería Pandas 🐼

Pandas es una herramienta de manipulación y análisis de datos de código abierto, rápida, potente, flexible y fácil de usar, construida sobre el lenguaje de programación Python.

pandas

La librería Pandas 🐼

Soporta archivos en formato:

- .xlsx, .xls, …
- .csv
- .tsv
- .json

y otras más …
(https://pandas.pydata.org/docs/user_guide/io.html)

La librería Pandas: DataFrames

pandas.DataFrame

class pandas.DataFrame(*data=None, index=None, columns=None, dtype=None, copy=None*)

Two-dimensional, size-mutable, potentially heterogeneous tabular data. [source]

Data structure also contains labeled axes (rows and columns). Arithmetic operations align on both row and column labels. Can be thought of as a dict-like container for Series objects. The primary pandas data structure.

La librería Pandas: DataFrames

index

nombre columnas

tuplas

	name	pop	pib
0	Metropolitana	7112808	24850
1	Valparaiso	1815902	14510
2	Biobío	1538194	13281
3	Maule	1044950	12695
4	Araucanía	957224	11064
5	O'Higgins	914555	14840

La librería Pandas: DataFrames Joins (Metodo Merge)

pandas.DataFrame.merge

```
DataFrame.merge(right, how='inner', on=None, left_on=None, right_on=None,
left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'), copy=True,
indicator=False, validate=None)
```
[source]

Merge DataFrame or named Series objects with a database-style join.

A named Series object is treated as a DataFrame with a single named column.

The join is done on columns or indexes. If joining columns on columns, the DataFrame indexes *will be ignored*. Otherwise if joining indexes on indexes or indexes on a column or columns, the index will be passed on. When performing a cross merge, no column specifications to merge on are allowed.

La librería Pandas: DataFrames Query

pandas.DataFrame.query¶

`DataFrame.query(expr, inplace=False, **kwargs)` [source]

Query the columns of a DataFrame with a boolean expression.

Parameters: **expr** : *str*

The query string to evaluate.

You can refer to variables in the environment by prefixing them with an '@' character like `@a + b`.

You can refer to column names that are not valid Python variable names by surrounding them in backticks. Thus, column names containing spaces or punctuations (besides underscores) or starting with digits must be surrounded by backticks. (For example, a column named "Area (cm^2)" would be referenced as `` `Area (cm^2)` ``). Column names which are Python keywords (like "list", "for", "import", etc) cannot be used.

For example, if one of your columns is called `a a` and you want to sum it with `b`, your query should be `` `a a` + b ``.

La librería Pandas: DataFrames Group By

pandas.DataFrame.groupby

```
DataFrame.groupby(by=None, axis=0, level=None, as_index=True, sort=True, group_keys=True,
squeeze=NoDefault.no_default, observed=False, dropna=True)
```
[source]

Group DataFrame using a mapper or by a Series of columns.

A groupby operation involves some combination of splitting the object, applying a function, and combining the results. This can be used to group large amounts of data and compute operations on these groups.

Data Engineering

Data Engineering

¿Son los conceptos anteriores todos los que necesitamos manejar para construir un sistema que maneja bien los datos?

Data Engineering

En algunos casos si …

Pero a medida que pasa el tiempo, la complejidad de los sistemas aumenta y en algunos casos también la carga.

Esto impone nuevos requisitos sobre las bases de datos, los cuales suelen requerir esfuerzos de ingeniería dedicados para poder hacerlos funcionar

Data Engineering: ¿Qué es?

- Rama de la ingeniería de software dedicada al desarrollo y mantenimiento de software para transmisión, procesamiento, almacenamiento y acceso de los datos.

- El rol de los ***data engineers*** en una organización suele ser proveer al resto de los miembros de plataformas en donde puedan acceder a los datos de forma confiable y eficiente.

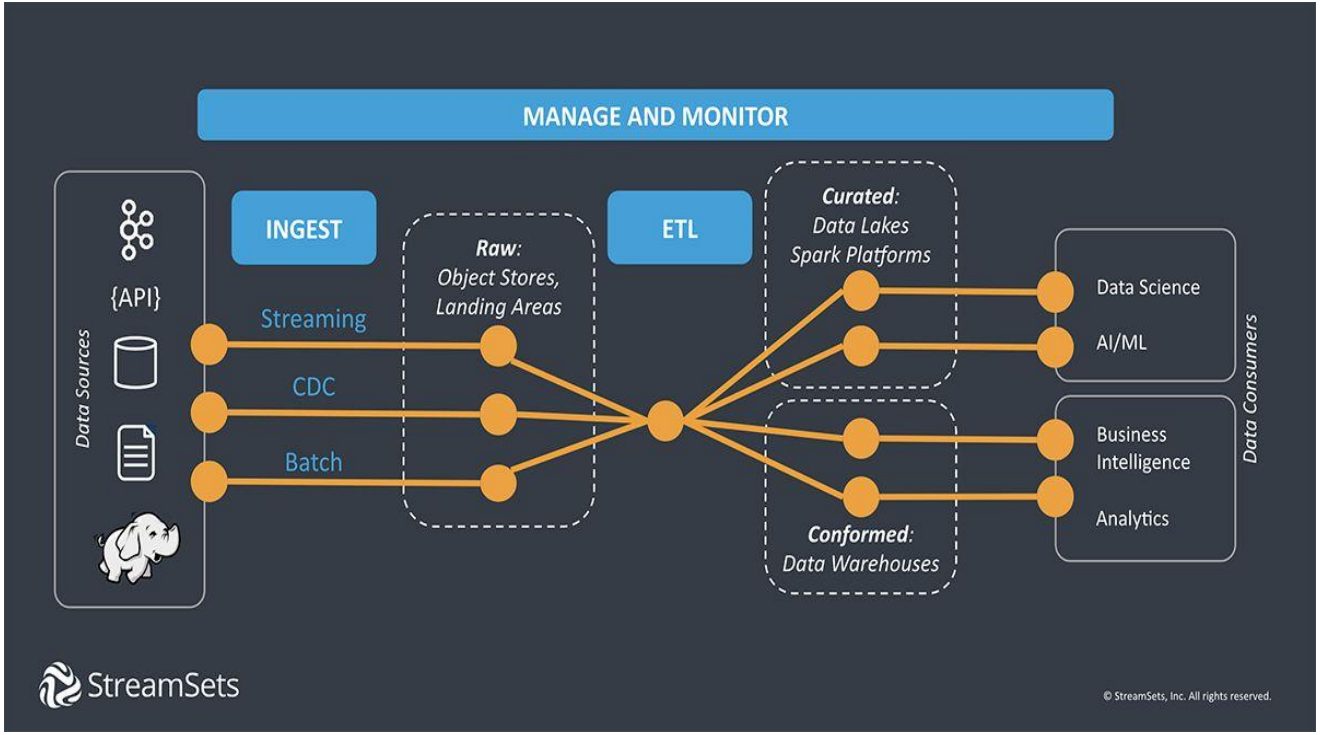
Data Engineering: ¿Por qué?

- Cuando los sistemas (y las organizaciones) crecen, nace la necesidad de generar protocolos e infraestructura para mantener los datos consistentes y accesibles 24/7, en distintos formatos.

- Distintos stakeholders necesitan consumir los datos de distinta manera: ***data scientists, business intelligence, developers,*** *etc.* Todos necesitan acceder a los datos de distinta forma y no pueden estar todos consultando la bd SQL principal.

- Todos los datos generados por un sistema tienen que ser extraídos, procesados y guardados según como los objetivos de la organización lo vayan requiriendo.

Data Engineering: Ejemplo de un sistema



Data Lakes & Data Warehouses

- Aplicaciones que funcionan basadas en una BD relacional suelen ejecutar muchas consultas tanto de escritura como de lectura, pero de baja complejidad. Se hacen pocos joins y agregaciones.
- Pero miembros de la organización que quieran analizar lo que ocurra en el sistema harán consultas con mucha agregación y joins. Cosas del tipo "Cuánto ha vendido en promedio cada tienda cada mes por los últimos 12 meses".
- Estas consultas son muy costosas para el DBMS y pueden afectar las disponibilidad de la aplicación para los usuarios reales!

Data Lakes & Data Warehouses

- A este tipo de consultas generalmente se les llama *On Line Analytic Processing* **(OLAP)***,* y queremos que se hagan en otro lado.

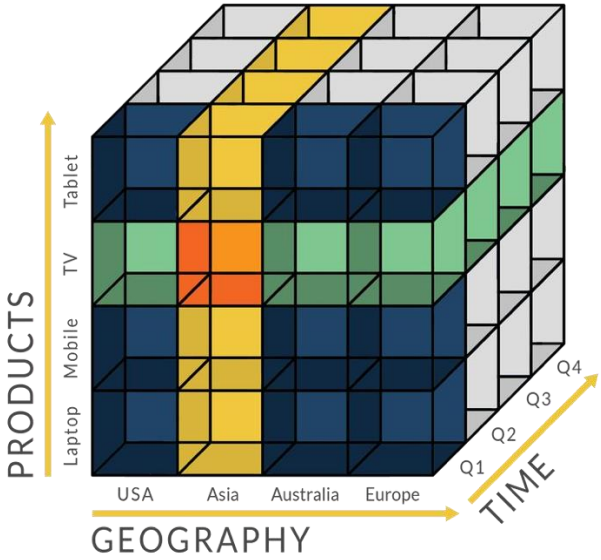
- Para eso se generan copias de la(s) bases de datos del sistema, ya sea tal cual (*Data lake*) o pre procesada para su uso de una manera particular (*Data warehouse)*.

- Estas copias no suelen estar actualizadas en tiempo real, pero se pueden consultar libremente y suelen mantener toda la información histórica del sistema (incluso aquella que se deprecó de la BD original)

Data Warehouses: OLAP Cube

- Un warehouse suele tener los datos preprocesados de alguna forma que facilite las consultas que se van a hacer en él.

- Una estrategia clásica es guardar los datos de forma “multidimensional”.



Data Warehouses: Columnar Databases

- Tablas que guardan los datos en disco columna a columna y no fila por fila.
- Se benefician de una alta paralelización (la nube lo hace solo).
- Se consultan con SQL.
- Algunas se pueden particionar para acelerar consultas específicas.

Google BigQuery

amazon REDSHIFT

ETLs: *extract, transform, load*

- Son scripts o tareas que toman datos desde una o más fuentes, la procesan y transforman según sea necesario y finalmente la guardan en otro lugar (usualmente un warehouse).

- Los ETLs suelen programarse para ejecutarse de manera periódica (cada 1 hora, 1 día, etc). En algunos casos se hacen *event-driven,* para procesar datos que vengan de un *stream.*

- Son una parte muy importante de cómo se procesan los datos en una organización.

Data Engineering en tiempos de la nube

Para la implementación de ETLs, data warehouses, lakes, streams de datos, etc. existen herramientas tanto open source como de proveedores cloud (pagadas).

El trabajo de data engineering en una organización grande suele consistir en combinar las herramientas open source con los servicios en la nube para generar pipelines para el procesamiento, guardado y uso de los datos.

Ejemplos open source

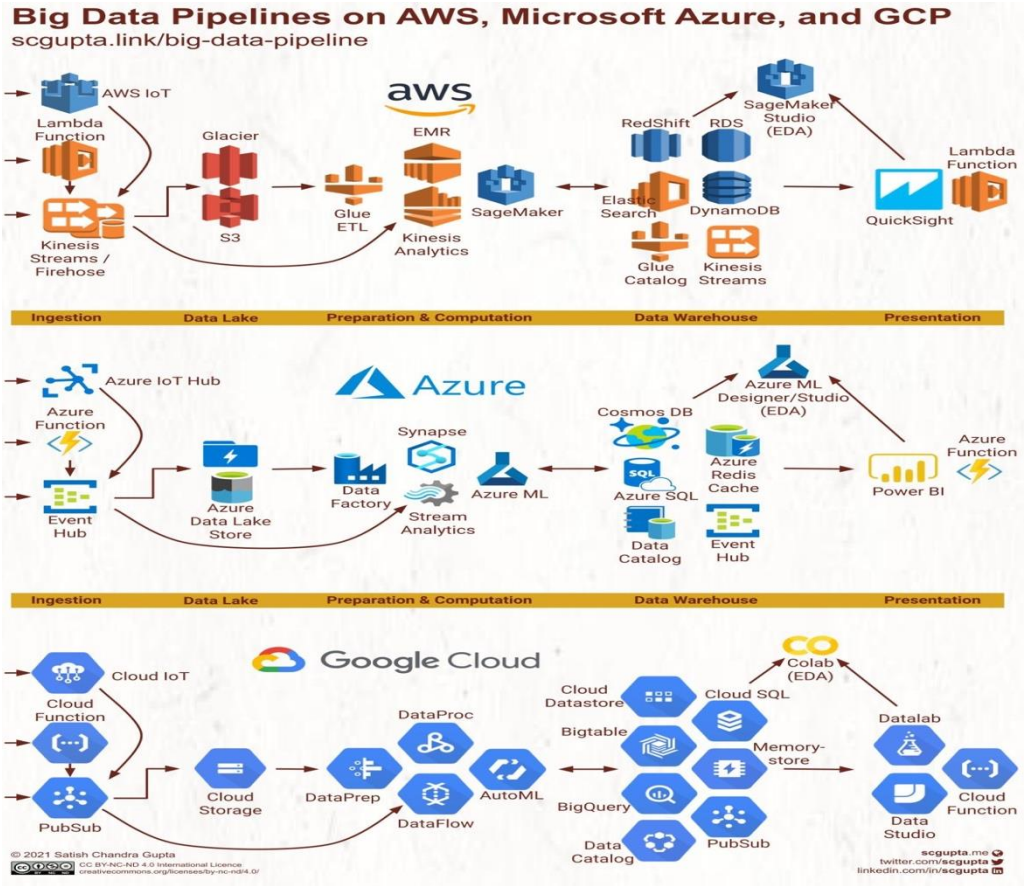
Plataforma para streaming distribuido de datos.

Procesamiento distribuido de datos (como map reduce)

Orquestación y ejecución de ETLs.

Procesamiento y analytics para datasets grandes.

Al final, un sistema de bases de datos complejos se ve así:



Veamos esto en un jupyter notebook…

Links para los curiosos

- https://www.intermix.io/blog/14-data-pipelines-amazon-redshif t/
- https://www.freecodecamp.org/news/scalable-data-analytics-p ipeline/
- https://aws.amazon.com/blogs/big-data/aws-serverless-data-a nalytics-pipeline-reference-architecture/
- https://www.slideshare.net/legoboku/building-a-data-pipeline-u sing-apache-airflow-on-aws-gcp

¿Dudas?

Si no,

FIN